standalone 宏包与文档类

Martin Scharrer
martin.scharrer@web.de

版本 v1.5a ——2025/02/22

CTAN 主页：https://www.ctan.org/pkg/standalone

版本库：https://github.com/MartinScharrer/standalone

摘要

standalone 宏包集合让用户可以方便地把图片环境或其他内容放进独立的源文件中，既能单独编译这些文件，也能把它们作为主文档的一部分来编译。为此提供了一个专门的 standalone 文档类，它在默认设置下会把输出文件裁切到内容大小。standalone 宏包则使用户可以在主文档中用 \input 直接读入这些 standalone 文件。

目录

1 在线资源与帮助 **2**

2 安装 **2**
2.1 在 TeX Live 中安装 . 2
2.2 在 MikTeX 中安装 . . 2
2.3 从 CTAN 手工安装 . . 2
2.4 依赖项 3

3 缺陷报告、功能请求与其他反馈 **3**

4 引言 **4**
4.1 快速上手 4
4.2 版本更新与向后兼容性 4
4.3 类似的宏包与文档类 . 5

5 standalone 文档类的用法 **6**
5.1 基本用法 6
5.2 类选项 6
5.3 宏与环境 10
5.4 对 Beamer 演示文稿的支持 11
5.5 类配置文件 13
5.6 简单的 TeX 文件 . . . 13
5.7 常见问题 / 故障排查 . 13

6 转换为图像 **15**
6.1 转换设置 15
6.2 转换软件 15
6.3 转换流程 18
6.4 转换示例 18

7 standalone 宏包的用法 **20**
7.1 基本用法 20
7.2 宏包选项 20
7.3 宏 23
7.4 从 standalone 文件构建图像 24

8 共用的宏 **24**

9 用法示例 **26**

1 在线资源与帮助

本宏包集合发布在全面的 TeX 宏包档案网（CTAN）上：https://ctan.org/pkg/standalone。不过任何现代的 LaTeX 发行版都会自动从那里获取它。CTAN 现在允许对宏包投票和评论，这类反馈非常欢迎。

本宏包集合的源代码托管在 Github 上：https://github.com/MartinScharrer/standalone。那里的工单系统应当用来报告缺陷或请求新功能：https://github.com/MartinScharrer/standalone/issues。

需要帮助的用户应当先在 TeX.Stackexchange.com 站点上搜索已有的解答：https://tex.stackexchange.com/questions/tagged/standalone 或者发布一个新问题：https://tex.stackexchange.com/questions/ask?tags=standalone。本宏包集合的作者是 TeX.SE 上的活跃用户，不过其他用户也可能帮得上忙。

2 安装

本宏包集合是两大主流 LaTeX 发行版 *TeX Live* 与 *MikTeX* 的一部分，可以用它们的包管理器方便地安装。它在 CTAN 上也以 TDS ZIP 文件的形式提供，可用于手工安装。此外，如小节 2.4 所述，还需要其他宏包。

2.1 在 TeX Live 中安装

使用普通的 TeX Live 时，借助包管理器 `tlmgr` 即可方便地安装本宏包集合。命令 `tlmgr install standalone` 会安装它，`tlmgr update standalone` 则可用来更新它。

由于 Ubuntu/Debian 版的 TeX Live 可能不含 `tlmgr`，因此需要手工安装该宏包。作者建议改为手工安装 TeX Live 的原版（vanilla）发行版，它会包含定期的宏包更新。

2.2 在 MikTeX 中安装

如 http://docs.miktex.org/manual/pkgmgt.html 所述，可以用 MikTeX 的包管理器方便地安装本宏包集合。MikTeX 中的宏包名称与普通宏包名称相同。

2.3 从 CTAN 手工安装

本宏包集合也在全面 TeX 档案网（CTAN）上提供，既有 DTX/INS 文件形式，也有可用于手工安装的 TDS ZIP 文件形式。只有所用的发行版不包含（所需版本的）该宏包时，才需要手工安装。

2.3.1 从 TDS ZIP 文件安装

TDS（TeX 目录结构）ZIP 文件中包含所有宏包文件的最终形式及其相对位置。它可以从 CTAN 下载：http://mirrors.ctan.org/install/macros/latex/contrib/standalone.tds.zip，也可以从 Github 项目站点下载：https://github.

com/MartinScharrer/standalone/releases/。只需把它解压到某个 TEXMF 目录中。在 Linux/Unix 下这通常是用户 TEXMF 目录 `~/texmf`。在 Windows 下通常是 `C:\Users\<用户名>\texmf`(Windows Vista/7)或 `C:\DocumentsandSettings\<用户名>\texmf`(Windows XP)。在 Mac OS X 下通常是 `/Users/<用户名>/Library/texmf`。另外也可以使用系统本地目录，它通常由环境变量 `TEXMFLOCAL` 给出。把文件复制到该位置之后，可能还需要更新 TEX 的文件名数据库。在 TeX Live 中可以通过运行 `texhash` 〈路径〉或 `mktexlsr` 〈路径〉来完成。MikTeX 提供了图形界面来刷新其文件名数据库，如 http://docs.miktex.org/manual/configuring.html#fndbupdate 所述。

2.3.2 从 DTX 文件安装

该宏包也以 DTX (Documented TeX file, 带文档的 TeX 文件) 形式提供, 并附有一个 INS (Install, 安装) 文件，位于 http://www.ctan.org/tex-archive/macros/latex/contrib/standalone。要从 DTX 文件中解包出所有宏包文件, 请用 `tex` 编译一次 INS 文件。手册可以用 `pdflatex` 从 DTX 文件中编译得到，这要求安装 `ydoc` 宏包集合。

2.4 依赖项

`standalone` 文档类与宏包需要 `xkeyval` 宏包。宏包 `ifpdf`、`ifluatex`、`ifxetex` 与 `shellesc` 若有则会被载入，否则会使用一些后备代码。若启用了类选项 varwidth、preview 与 beamer，则需要同名宏包或文档类。

`standalone` 宏包需要 `currfile` 宏包（后者又用到 `filehook`），以便正确追踪用`\input` 读入的子文件的文件名。若要对读入的 standalone 文件提供编译支持，还需要 `gincltex` 与 `filemod` 宏包。

编译 `standalone` 的文档需要 `ydoc` 宏包集合。

上述宏包都包含在较新版本的 TeXLive 或 MikTeX 发行版中，并且可以在 CTAN 上免费获取。

3 缺陷报告、功能请求与其他反馈

关于 `standalone` 宏包集合的缺陷报告、功能请求与其他反馈, 可以通过下列方式之一发送给作者：用电子邮件发至 martin.scharrer@web.de，或使用该宏包集合的工单系统 https://github.com/MartinScharrer/standalone/issues。缺陷报告应包含所用的 `standalone` 版本、所用的 LATEX 格式（`pdflatex`、`latex`、`xelatex` 等）以及发行版及其版本号。通常，一个能重现该问题的最小工作示例对分析和解决缺陷有极大的帮助。请先查找已有的相关工单，并先查阅小节 5.7 中的常见问题/故障排查。与 preview 类选项相关的问题，应与直接使用底层 `preview` 宏包的情形作对比。

4 引言

较大的 LaTeX 文档可以拆分成多个 TeX 文件，再用 \include（例如各章文件）或 \input（例如 TeX 代码绘制的图片）把它们读入主文档。把图片放在各自的子文件中能提高主文件的可读性，也便于在不同文档之间共享它们。然而，在有时相当漫长的绘制/编码过程中，能够单独编译这些图片是有好处的。编译过程要快得多，而且得到的文档只包含这张图片，避免了一直翻页和缩放。

虽然可以为每个图片文件写一个小的“主”文件，但这种方法略显笨拙，还会让目录中堆满许多多余的文件。第二种方法则是把“主”成分（即导言区）直接放进图片文件中，并让主文档忽略这些代码段。

在主文档中可以使用 standalone 宏包来跳过被读入文件中的所有额外导言区。主文件必须载入子文件所需的所有宏包与设置。宏包提供了若干选项，用于自动收集子文件的导言区并在主文件中执行它们。

另外还提供了 standalone 文档类，以尽量减少这些文件中所需的额外导言区代码。它的使用是可选的，但能简化并统一图片文件单独编译的方式。该类默认使用 crop 选项，生成只包含图片、没有多余页边距、页码或其他任何内容的输出文件。该类会读入一个配置文件 standalone.cfg，用户可以用它按目录方便地调整设置和宏。

4.1 快速上手

在主文档中要尽早载入 standalone 宏包。同时，所有子文件需要的宏包也都必须由主文档载入。像平常一样用\input 或类似的宏读入你的图片或其他子文件。在子文件中用普通的 \documentclass 使用 standalone 文档类，并载入该文件所需的全部宏包。最后把子文件的实际内容包裹在 document 环境里。避免在文档主体的开头或结尾出现空行。

当子文件单独编译时，\documentclass 与 document 环境会像平常一样生效。而主文件则会跳过从\documentclass 到\begin{document} 的所有内容。（此时已属“假”的）document 环境被重定义为一个简单的 TeX 分组。\end{document} 之后的任何代码行都会被忽略。主文件真正的 document 环境不受影响，会照常工作。

各子文件所需的宏包可以通过小节 7.2 中所列的选项自动转移到主文档的导言区。

4.2 版本更新与向后兼容性

standalone 文档类 v1.x 的默认行为与 v0.x 略有不同，但对大多数 standalone 文件而言，输出结果应当相同。在以前的版本中，preview 选项是默认启用的，而自 v1.0 起改用新的、与之类似的 crop 选项。这一变化应当能改善若干用例，例如避免因末尾空行而产生一个段落，以及 XeLaTeX 下 TikZ 图案的问题。不过现在默认会忽略段落分隔，对于图片类以及 standalone 文档类主要针对的类似环境来说，这应该完全不成问题。另外，默认边界已从 preview 的默认值 0.50001bp 改为无边界（0pt）。这两项设置都可以通

过在配置文件中加入 `\standaloneconfig{preview,border=0.50001bp}`，或把这些选项明确地作为类选项写出来，从而改回旧的默认值。

v0.x 与 v1.x 之间一处真正的不兼容是类配置文件的载入时机。在 v0.x 中，配置文件在所有选项处理完毕之后才载入，以便所有 if 开关都取得最终值。在 v1.x 中，配置文件改为在给定的类选项被处理之前直接载入。这样可以方便地为所有 standalone 文件设置默认选项。依赖 if 开关（如 `\ifstandalone` 与 `\ifstandalonebeamer`）的代码应放进 `\AtEndOfClass{`⟨ 代码 ⟩`}` 宏中。这一变化可能需要更新个人的配置文件。

4.3 类似的宏包与文档类

下列宏包、库和/或文档类针对与本 宏包集合相同或类似的用途，这里列出以便比较，让用户能决定哪一个最适合自己。

`docmute` 宏包与 `standalone` 宏包处理的基本任务相同。但它除移除之外不支持任何子导言区处理。它也不提供专门的文档类或配置文件。

`subfile` 宏包与文档类也是为了让子文件能单独编译。然而 `subfile` 文档类会从给定的主文件导入导言区，而 `standalone` 的设计方向与此相反：子文件的导言区可以被导入到主文档中。因此 `standalone` 文件更容易被多个文档共用，例如一篇论文（科学出版物）、对应的演示文稿，再到一篇学位论文；而 `subfile` 是为一一对应的关系设计的。在撰写本文时，`subfile` 由于缺少许可证声明而未包含在 TeXLive 中。

`tikz` 的 `external` 库允许把 `tikzpicture` 从主文档中外置。它的构建功能与 `standalone` 提供的功能类似。不过二者的方向不同：`standalone` 允许把外部的 `tikzpicture` 读入主文件并忽略其导言区，而 `external` 则把它们从主文件写出到临时的外部文件。用户必须自行决定哪种工作流程更适合自己。此外，`standalone` 独立于 `tikz` 工作，并支持其他图片环境，如 `pstricks` 或任何其他 TEX 内容。

5 standalone 文档类的用法

5.1 基本用法

创建一个基本的 standalone 文件很直接：写一个使用 `standalone` 作为文档类的普通 LaTeX 文档。导言区应载入内容所需的所有宏包与库。内容通常是一个图片环境，如 `tikzpicture`，放在 `document` 主体中。应避免图片前后的空行。同时`\begin{document}` 与`\end{document}` 应各自独占一行源代码。

清单 1: 基本用法：`standalone` 文档类。

```
\documentclass{standalone}
\usepackage{somepackage}
\begin{document}
\begin{somepicture}
  \somedrawingcommands
\end{somepicture}
\end{document}
```

这样的文件可以正常编译。`standalone` 文档类会把得到的输出文件（PDF 或 DVI/PS）裁切到内容尺寸再加上一定的边界。页码以及其他页眉页脚材料都会被抑制。

对于用 TikZ 绘制的图片，提供了一个专门的 tikz 选项，它会载入 `tikz` 宏包，并把 `tikzpicture` 环境配置为生成单个裁切后的页面。对于 PSTricks 图片，则有相应的 pstricks 选项。

清单 2: 基本用法：`standalone` 文档类。

```
\documentclass[tikz]{standalone}
%\usetikzlibrary{calc}
\begin{document}
\begin{tikzpicture}
  \draw (0,0) rectangle (2,1) node [midway] {示例};
\end{tikzpicture}
% 还可以有更多 'tikzpicture' 环境，它们会再生成其他页面。
\end{document}
```

5.2 类选项

`standalone` 文档类提供下列选项，用于调整内容的处理方式与尺寸。这些选项会从普通的类选项列表中移除，并且不会像通常那样传给任何被载入的宏包或文档类。这样做也是为了避开与底层文档类同名选项的冲突。

所有布尔选项都以 '`true`' 或 '`false`' 作为可选值。若选项不带值使用，则采用 '`true`'。除非另有说明，所有选项初始都设为 '`false`'。选项可能会开启或关闭其他选项。例如互斥的选项会互相禁用。选项的先后顺序会被遵

守，后面的选项会压过前面的选项。

默认启用 crop 选项且 border=0。在 v1.0 之前的版本中，默认选项是 preview。这一改动被认为是必要的，应当不会影响大多数文档。不过在有些情况下，可能仍需重新启用 preview 选项。

某些类选项也可以在导言区或文档主体中用 `\standaloneconfig{⟨ 选项 ⟩}` 设置。

`class=⟨ 类名 ⟩`

指定由 standalone 文档类载入的底层文档类。默认使用 article，它应当适合 standalone 图片。在某些情况下，使用与目标主文档相同的文档类可能更有好处。对于 beamer 文档类，应改用专门的 beamer 选项。

`crop=true|false`

若启用，该选项会把内容裁切到其自然尺寸再加上指定的边界。做法是把内容存入一个盒子寄存器，并相对于盒子的尺寸调整页面大小。该选项默认启用（自 v1.0 起）。该选项与类似的 preview 选项互斥，因此会禁用后者。若两个选项都被使用，则后面那个被启用而另一个被禁用。另外，`crop=true` 会设置 `float=false`，以避免浮动环境带来的问题。

`preview=true|false`

若启用，该选项会带着 tightpage 选项载入 preview 宏包，并把内容包裹进 preview 环境。这会把内容裁切到其自然尺寸再加上指定的边界。有报告指出 XeLaTeX 下 preview 选项与 TikZ 阴影（shading）存在问题。这种情况下应改用 crop 选项。请注意，该选项在 v1.0 之前的版本中是默认启用的，但此后默认启用的已改为 crop。

该选项与类似的 crop 选项互斥，因此会禁用后者。若两个选项都被使用，则后面那个被启用而另一个被禁用。另外，`preview=true` 会设置 `float=false`，以避免浮动环境带来的问题。

```
border=⟨ 长度（四边）⟩
border={⟨ 长度（左/右）⟩ ⟨ 长度（下/上）⟩}
border={⟨ 长度（左）⟩ ⟨ 长度（下）⟩ ⟨ 长度（右）⟩ ⟨ 长度（上）⟩}
```

该选项用于指定 preview 与 crop 选项所使用的边界。该选项的另一个名称是 margin。边界既可以对所有边给出一个值，也可以分别给出水平与垂直边界的值，或分别给出各边的值。多个值之间用空格分隔，这就要求把整个值用花括号包起来。默认设置的边界为 0pt。

该选项可以在文档中用 `\standaloneconfig{⟨ 选项 ⟩}` 更改，并会影响其后的所有页面。

`multi=true|false`
`multi={⟨ 环境名 ⟩, …}`

默认情况下，standalone 文档类假定整个内容是一个块，应显示在单独一页上。若启用该选项，则支持多页。每一页都会裁切到其内容再加上所选边界（只要 preview 或 crop 之一启用）。承载单页的一组环境必须以选项值的形式给出，或用 `\standaloneenv{⟨ 环境名 ⟩, …}` 声明。此类环境之外不应使用任何排版材料。请注意，若 crop 或 preview 启用，该选项会被 `\standaloneenv` 自动启用。不过，如果同时设置了 ignorerest 选项，就必须把它明确地作为类选项设置。若以选项值的形式给出环境名，该选项会被设为 'true'，而这些环境会被传给 `\standaloneenv`，后者在 document 环境开始时执行，那时上述所有环境应当已经定义。

`ignorerest=true|false`

该选项只在 multi 与 crop 都启用时才有意义。此时它决定是否忽略所有不出现在用 `\standalone` 声明的环境中的材料。做法是把所有外部材料装箱后丢弃。任何代码都会被放进一个分组中，因此在各环境之间所做的局部设置不会影响后面的代码。导言区中的代码不受影响。建议保持该选项禁用，只在确实需要时才使用。应当指出，使用 preview 时此类材料总会被忽略，且不影响局部设置。因此，如果用户需要，可以把 ignorerest 选项看作一种兼容性设置，让 crop 的行为更像 preview。

`multido=true|false`

人们常用 multido 宏包及其 `\multido` 宏来生成图形的多次迭代。通常每次迭代都应放在单独的一页上。该选项通过把 `\multido` 改成自动将内容（第三个参数）包裹进一个 multi 环境，来简化这项工作。该选项只有在 `\multido` 宏位于顶层、而不是像 pspicture 这样的环境的一部分时才有效。

multido 支持 ignorerest 选项，因为这里特别注意不去忽略这些 `\multido` 宏。

支持 `\multido` 宏的嵌套，只有外层会产生页面，内层则照常工作。对于任何 multi 环境内部的 `\multido`，情况也是如此。

该选项会设置选项 multi=samultido，载入 multido 宏包，并重定义其内部宏 `\multido@`。这样做是为了同时支持其他宏 `\mmultido`、`\Multido` 与 `\MMultido`。

`varwidth=true|false`
`varwidth=⟨ 宽度 ⟩`

内容与 `\end{document}` 之间的末尾空行通常会创建一个宽度为 `\linewidth` 的段落。这个段落（或任何其他段落）会增大较小图片的尺寸，表现为一个很宽的右边距。该选项使用 varwidth 宏包把内容包裹进 varwidth 环境，

后者基于 minipage，但当内容宽度小于给定的最大宽度时，总是采用内容的自然宽度。最终效果是，所产生的段落不会造成额外宽度，并且可以把多个段落作为内容的一部分。所用的最大宽度（它被传给底层的 minipage 环境）默认为 \linewidth，但可以通过给选项提供一个宽度值来设置。这样做同时也会打开该选项。

若使用了 crop 选项，内容会被放在忽略段落分隔的受限水平模式中。使用 varwidth 选项则重新启用段落分隔。

该选项的一个缺点是内容会被限制在给定宽度之内，即较宽的图片环境会在右侧被裁掉。这种情况下，应当选择更大的宽度、关闭该选项、避免任何段落分隔（不要有末尾空行），或改用 tikz、pstricks 之类专门的图片选项。

该选项可以在文档中用 \standaloneconfig{⟨ 选项 ⟩} 更改，并会影响其后所有页面的内容。

tikz=true|false

该选项声明内容由一个或多个 tikzpicture 环境组成。它会设置 multi=tikzpicture, varwidth=false 并载入 tikz 宏包。

pstricks=true|false

该选项声明内容由一个或多个 pspicture 或 pspicture* 环境组成。它会设置 multi=pspicture,varwidth=false 并载入 pstricks 宏包。由于 pspicture* 内部使用 pspicture，所以它也受到支持。其他同样使用它的环境应该也能得到支持，但可能还需要用 \standaloneenv{⟨ 环境名 ⟩, …} 明确声明。

beamer=true|false

若设为 'true'，该选项会启用一种特殊的 beamer 模式，其中正常的裁切被禁用。内容改为显示在一张空白的 beamer 幻灯片（frame）上。

float=true|false

若该选项设为 'false'（这是默认值），则 figure、table 等任何浮动环境都会变成非浮动环境。crop 与 preview 选项要正常工作需要这样做，所以它们自己设为 'true' 时会把 float=false。一般来说，建议把浮动环境留在主文档中，只把其内容放进 standalone 文件。这也能让同一内容方便地被不同主文档中的不同浮动体使用。

目前还不支持用 float 之类宏包定义的自定义浮动体。取决于它们定义浮动体的方式，或许仍能工作。对于这些情况，应把 float=true 作为类选项设置，以便保留浮动体的正常定义。之后再使用 \standaloneconfig{float=false} 禁用浮动体，同时把改动后的浮动体定义考虑进去。

```
convert={⟨ 转换选项 ⟩}
png={⟨ 转换选项 ⟩}
jpg={⟨ 转换选项 ⟩}
gif={⟨ 转换选项 ⟩}
svg={⟨ 转换选项 ⟩}
emf={⟨ 转换选项 ⟩}
```

这些选项用于启用和配置转换功能。完整说明见节 6。

5.3 宏与环境

下列宏与环境可以在 standalone 文件的导言区中使用。节 8 中列出了更多宏，它们由文档类与宏包共同定义，既可用于 standalone 文件，也可用于主文档。

```
\standaloneconfig{⟨ 选项 ⟩}
```

该配置宏接受小节 5.2 中描述的类选项。它可以用于类配置文件，以设置所有 standalone 文件使用的默认设置，如小节 5.5 所述。这些设置会在 standalone 文件的类选项被处理之前刚刚设置好。

某些没有全局效果的类选项（例如 border、varwidth）也可以稍后在导言区用该宏更改，甚至在启用了 multi 选项时，在文档主体中的不同内容之间更改。

```
\standaloneenv{⟨ 环境 ⟩,⟨ 环境 ⟩,…}
```

若 multi 选项生效，应使用该宏声明所有会产生内容的环境。这类环境的常见例子有 tikzpicture、pspicture 以及其他图片环境。该宏只能在导言区使用。文档主体中每次使用这样的环境都会产生新的一页。例外情况是这类环境的嵌套出现，例如一个 tikzpicture 位于另一个 tikzpicture 的节点中。这些环境必须事先已定义，并且之后不得重定义。应避免在同一个或多个 \standalonenv 中多次出现同一个环境名。

若 preview 选项生效，该宏内部使用 \PreviewEnvironment；若使用替代的 crop 选项，则使用自有代码。若这两个选项都没有启用，该宏将不起作用，并被静默忽略。

```
\standaloneignore
```

在少数情况下，必须把某些代码放在子文件的\documentclass 之前（例如\PassOptionsToPackage）。由于主文档只跳过\documentclass 与\begin{document} 之间的代码，这些代码会被它执行。为避免这一点，可以在子文件的最开头使用宏 \standaloneignore 跳过这段代码。不过它必须写成\csname standaloneignore\endcsname，以免单独编译时出现 Undefined control sequence 错误。毕竟此时文档类尚未载入，因此还没有定义任何 standalone 宏。这种情况下，\csname …\endcsname 结构只会使它等于\relax 。

请注意，\documentclass 之前的所有代码都不会被任何 subpreamble 选项处理，而总是被简单地删除。该宏受到了 docmute 宏包中类似的宏\docmute 的启发。

```
\begin{standalone}
  ⟨ 子文件内容 ⟩
\end{standalone}
```

standalone 环境会自动包裹在 standalone 文件的内容之外。若启用了 multi 选项，它会包裹在每一页之外，即每个用 \standaloneenv 声明的环境之外。该环境的定义取决于 crop 与 preview 等选项。可以在配置文件或文档导言区中重定义该环境，以调整内容的处理方式，但不推荐这样做。若这样做，大多数与内容相关的选项会停止工作，并且/或者导致错误。

beamer 专用的宏与环境在小节 5.4 中描述。

5.4 对 Beamer 演示文稿的支持

演示文稿可以用 LaTeX 配合 beamer 文档类来编写。每张幻灯片都包裹在 frame 环境中。叠加（overlay）效果可以用专门的宏添加。这些效果会导致一张幻灯片产生多页。带有这类叠加效果的图片无法用常规设置单独编译。此时 standalone 文档类必须载入 beamer 文档类，并同时把内容包裹进 frame 环境，而跳过 preview 环境。要启用这些设置，请带 beamer 选项载入 standalone 文档类。由于 frame 环境相当特殊（它通常会收集其全部内容并调用 \frame），而且必须支持逐字（verbatim）内容，因此把 document 环境重定义成包含 frame 并不容易做到。另外 frame 接受 document 不接受的选项。因此在 beamer 图片文件中使用第二个名为 standaloneframe 的环境。它在 standalone 模式下等同于 frame，在其他情况下则不起作用。

```
\ifstandalonebeamer
```

文档类与宏包都提供 if 开关 \ifstandalonebeamer，可用于只在文件带 beamer 类选项单独编译时才包含的代码。带 beamer 选项载入时，该开关被文档类设为 \iftrue，而被宏包总是设为 \iffalse。它可以用于在配置文件中放置 beamer 专用选项，这些选项对非 beamer 的 standalone 文件应当跳过。若在配置文件中使用，该开关必须放在 \AtEndOfClass{…} 内部，因为 beamer 选项此时尚未处理。

```
\begin{standaloneframe}<⟨ 叠加说明 ⟩>[<⟨ 默认叠加说明 ⟩>]
                  [⟨ 选项 ⟩]{⟨ 可选的幻灯片标题 ⟩}{⟨ 可选的幻灯片副标题 ⟩}
  ⟨ 带 beamer 叠加效果的代码 ⟩
\end{standaloneframe}
```

若子文件中含有 beamer 叠加代码，就必须使用 standaloneframe 环境。它只在带 beamer 选项调用文档类时才有定义，单独编译时充当 beamer 的 frame 环境的替代品。frame 的所有可选参数都受支持，但大多数对普通子

文件可能没什么用。当作为主文档的一部分编译时，它除了吞掉其参数之外什么也不做。

清单 3–5 展示了 beamer standalone 示例，以及它在 standalone 模式与主文档模式下实际生效的代码。

清单 3: `beamer` 选项配合 `standalone` 文档类。

```
% 配合 beamer 叠加效果使用 'standalone' 文档类:
\documentclass[beamer]{standalone}
% 载入本 TeX 文件需要的宏包:
\usepackage{tikz}
% 按通常做法把 TeX 代码置于 'document' 环境之中:
\begin{document}
\begin{standaloneframe}[<选项>] % 例如 'fragile'
 % 在此添加你的 TeX 代码:
  \only<1>{ One }%
  \only<2>{ Two }%
\end{standaloneframe}
\end{document}
```

清单 4: 单独编译时实际生效的 beamer 代码。

```
\documentclass{beamer}
<来自 standalone.cfg 文件的 beamer 代码>
\usepackage{tikz}
\begin{document}
\begin{frame}[<你的选项>]
  \only<1>{ One }%
  \only<2>{ Two }%
\end{frame}
\end{document}
```

清单 5: 被包含进 beamer 演示文稿时实际生效的代码。

```
\begingroup
  \only<1>{ One }%
  \only<2>{ Two }%
\endgroup
\endinput
```

5.5 类配置文件

在处理选项之前、但在所有选项与 if 开关声明之后，standalone 文档类会载入一个名为 standalone.cfg 的配置文件。之后任何类选项也都可以用 \standaloneconfig{⟨ 选项 ⟩} 给出。依赖最终所用选项的设置应放在 \AtEndOfClass{…} 内部，以便在所有选项之后处理。这对 beamer 专用设置尤其必要，因为在载入配置文件时，给定的 beamer 选项尚未处理。请注意，v1.0 之前对此的处理方式不同，因此在用户编辑过的旧配置文件中，现在必须加上 \AtEndOfClass 。

宏包集合自带一个默认配置文件，其中有一些默认设置。由于该文件在每次更新宏包集合时都会被覆盖，用户应在本地 TEXMF 树或文档目录中创建自己的配置文件。为保持默认行为，该文件应包含宏包集合配置文件的内容或载入它。由于可以假定宏包集合的配置文件位于一个 standalone 目录中，因此可以从用户配置文件用 \input{standalone/standalone.cfg} 载入它。

5.6 简单的 TeX 文件

宏包集合还附带一个简单的 standalone.tex 文件,在特殊场合可能有用。在单独编译时,它会把 \ifstandalone 开关设为 *true*;而在任何\documentclass 宏之后被载入时，只要该开关尚未定义，就设为 *false*。如果在 standalone 文件的\documentclass 之前需要这个开关，就必须使用它。

清单 6: 'standalone.tex' 的用法。

```
\input{standalone} % 放在任何 '\documentclass' 之前使用
\ifstandalone
  % 仅在单独编译时使用
\fi
```

5.7 常见问题 / 故障排查

本节展开说明使用 standalone 文档类时可能出现的一些问题及其解决办法。

右侧出现大片空白 / 边界

当内容被放在一个段落中时，右侧就会出现大片空白 / 边界。这会使内容宽度变为 \linewidth ，于是较小的图片右侧会有空白。常见原因是内容与 \end{document} 之间有一个空行，从而造成段落分隔。

该问题可以通过删除任何末尾空行或其他段落分隔来解决，或使用抑制额外宽度的 varwidth 选项。也可以使用 multi 选项和 \standaloneenv{⟨ 环境名 ⟩} 把某些环境声明为页面内容。tikz 选项对 tikzpictures 这样做，pstricks 选项则对 pspicture 这样做。详见这些选项的说明。

右侧有一部分内容缺失

如果内容在右侧被裁得过多，请检查是否用了 varwidth 选项。此情况下所用的最大宽度（默认 `\linewidth`）太小。可以用varwidth=⟨ 长度 ⟩ 设置更大的宽度，或用varwidth=false 完全禁用该选项。可能的最大宽度由 `\maxdimen` 给出，但这可能导致内部溢出。

这也可能由 beamer 内容引起（即使用 beamer 选项时）。此情况下完全不做裁切，也不使用 varwidth 环境，而只是内容太大，放不进一张 beamer 的 frame。为避免这一点，请把内容重新缩放使其放得下。这可以通过所用图片环境的缩放功能（如 environment 的 scale，但它只缩放坐标），或 graphicx 的 `\scalebox` 或 `\resizebox` 来实现。对于含逐字内容或其他改变 catcode 的代码的复杂代码，应使用 realboxes 宏包的 `\Resizebox` 或 adjustbox 宏包的 {adjustbox}{scale=⟨ 缩放因子 ⟩} 环境。

多页文档中有一些页面包含不需要的内容

当设置了 multi=true 与 crop=true,但 ignorerest=false 且文档在用 `\standaloneenv` 声明的环境之外含有排版材料时，就会出现这种情况。为避免它，应删除这些额外材料，或把 ignorerest 设为 true。这也会忽略文档主体中所有不在已声明环境内的设置。可以把这些设置移到导言区。详见 ignorerest 选项的说明。

使用 DVI/PS 模式的多页文档中，除第一页外所有页面都有竖直偏移

PostScript 中的竖直参考点不会随页面被调整为各页各自内容的大小而改变。因此会加上一个偏移量来补偿，从而把内容移到合适的竖直位置。若这不能按预期工作，请告知宏包作者，并提供能引起该问题的小示例，以及所用的 latex 编译器与工具（如 dvips、ps2pdf）的版本号，还有所用的 standalone 宏包集合版本。

DVI 模式下裁切文件的问题

crop 选项在 DVI 模式下使用 PostScript 命令，即当以 latex 而不是 pdflatex（或其他）作为编译器时。这些 PostScript 命令只有在 DVI 被转换成 PS 或 EPS 之后才会起作用。目前这段裁切代码是实验性的，可能不会产生完全符合 (E)PS 标准的文件。这可能导致边界框（bounding box）错误、方向错误，或者取决于所用的 PostScript 工具，甚至出现 PostScript 编译错误。其中一些问题可以通过用 eps2eps 或 Ghostscript 等工具把 (E)PS 文件转换成（更）符合标准的版本来克服。

报错“Float(s) lost”或“not in outer par mode”

当 `float=true` 且设置了 `crop=true` 或 `preview=true` 时,不能使用 figure 或 table 之类的浮动环境。后两个选项会试图把浮动体存进一个盒子，这是不允许的（因为它再也无法保持浮动）。通常 `float=false` 能解决这个错

误，因为它把这些环境变成非浮动的替代品。由于 `crop` 与 `preview` 选项自身都会设置 `float=false`，只有在此后手动设置 `float` 选项时才会出现该问题。

图像转换不能工作

要让图像转换工作，必须安装一个外部转换软件。默认使用 Image Magick 或 GhostScript 之一。请确保其中之一或两者都已安装。适用于你所用操作系统的安装指南在网上应当很容易找到。必须给 LaTeX 编译器使用`-shell-escape` 选项，以便允许从 LaTeX 代码中执行该外部软件。若这两点都满足而转换仍不能工作，请查看日志文件。相关行以 '`runsystem`' 开头（至少在 TeX Live 2011 中如此）。

6 转换为图像

使用 convert 类选项，standalone 文件可以方便地转换成光栅图像。做法是执行一个外部程序，把输出文件（PDF 或 PS）转换成图像（推荐使用无损的 PNG 格式，但也支持其他格式）。

6.1 转换设置

转换设置可以作为 `convert={`⟨ 转换设置 ⟩`}` 选项的值给出。默认转换是禁用的（`convert=false`）。若启用但未提供自己的设置（`convert`、`convert=true`），则使用以下默认设置：PNG 格式、300dpi 的密度、不指定明确尺寸、输出文件名由 `\jobname` 给出，即 LaTeX 文档的名字。用 convert 选项并以任何非 `false` 的值使用它都会启用转换。所有常规转换设置列在表 1 中，而表 2 列出了更高级的选项，例如可直接修改转换命令的选项。注意，documentclass 选项内部使用的宏必须防止展开。可以把整个参数包裹进 `\unexpanded{..}`（需要 e-TeX），或在任何宏前使用 `\noexpand` 来做到。

6.2 转换软件

转换需要一个已安装的外部图像转换程序。支持以下两个工具。要执行外部程序，必须给编译器可执行文件使用 `-shell-escape` 选项[1]，例如 `pdflatex -shell-escape filename`。没有这个选项就无法转换，因为 LaTeX 会阻止对 shell 的调用。

默认使用转换程序 Ghostscript，它是一个非常常见的 PostScript 解释器，也支持 PDF。在 Ubuntu Linux 下它很可能已安装，否则可以用 '`sudo apt-get install ghostscript`' 或 '`sudo apt-get install gs`' 安装。它既能转换成各种输出格式，又可在 Unix/Linux、Max OS X 与 MS Windows 上免费获取。它需要设置正确的输出设备，而输出设备并不总是与输出格式完全一致（例如 PNG（含 1600 万色）用 '`png16m`'）。PNG 与 JPG 的设备已预先配置好。其他设备可以用 `defgsdevice={`⟨. 扩展名 ⟩`}{`⟨ 设备 ⟩`}` 转

[1] 按所用的 LaTeX 发行版，名称可能不同

表 1: 转换选项（用于convert 类选项的值）

子选项	说明	默认值
（无值）	用默认设置启用转换	./.
true	启用转换（若未给出其他选项则使用默认设置）	（无值）
false	禁用转换	（无值）
png	选择 PNG 输出格式	
jpg	选择 JPG 输出格式	
gif	使用 *ImageMagick* 选择 GIF 输出格式	
svg	使用 *Inkscape* 选择 SVG 输出格式	
emf	使用 *Inkscape* 选择 EMF 输出格式	
density	以每英寸点数（dpi）设置密度。可以是单个数值，也可以是 '⟨*X*⟩x⟨*Y*⟩'。	300
size	设置图像的尺寸。可以是单个数值，也可以是 '⟨*X*⟩x⟨*Y*⟩'。若为空，尺寸由密度设置和 PDF 的大小决定。	（空）
subjobname	内部 LaTeX 运行所用的 jobname	\jobname
inext	输入文件的扩展名（含前导的点）	.pdf 或.ps
inname	输入文件的名称基（即不含扩展名的文件名）	\subjobname
infile	输入文件名	\indir\inname\inext
outext	输出文件的扩展名（含前导的点）	.png
outname	输出文件的名称基	\inname
outfile	输出文件名	\outdir\outname\outext
outputdir	用于 LaTeX 编译器的-output-directory 选项的目录。这会使内部运行产生的所有 LaTeX 输出文件都在给定目录中创建并从中读回。若主 LaTeX 编译器运行时使用了-output-directory 选项，就应设置该选项，但也可以单独使用。若不为空，该目录必须以斜杠 '/' 结尾。	（空）
outdir	转换结果应当生成于其中的输出目录。若不为空，该目录必须以斜杠 '/' 结尾。	\outputdir
indir	输入目录，例如 PDF 生成所在的目录。若不为空，该目录必须以斜杠 '/' 结尾。	\outputdir

注意：这些设置（'true' 与'false' 除外）也可以在其他设置中作为宏使用。

表 2: 高级转换选项

子选项	说明	默认值
command	用于转换的命令行。	（见 imagemagick）
perpage	布尔值。若设为 true（不带值使用时的默认值），则转换命令对每一页调用一次。需要启用 multi 选项。	
imagemagick	把转换命令设为使用 Image Magick：	
	`command=\unexpanded{{\convertexe\space -density \density\space \infile\space \ifx\size\empty\else -resize \size\fi\space -quality 90 \outfile}}`	
convertexe	Image Magick 可执行文件的名称。	（见小节 6.2）
ghostscript	把转换命令设为使用 Ghostscript：	
	`command=\unexpanded{{\gsexe\space -dSAFER -dBATCH -dNOPAUSE -sDEVICE=\gsdevice\space -r\density\space -sOutputFile=\outfile\space \infile}}`	
ghostscript32	把转换命令设为使用 Ghostscript，并把 gsexe 设为 32 位可执行文件。	
ghostscript64	把转换命令设为使用 Ghostscript，并把 gsexe 设为 64 位可执行文件。	
gsexe	Ghostscript 可执行文件的名称。	（见小节 6.2）
gsdevice	供 Ghostscript 使用的输出设备。针对 PNG 与 JPG 输出已预先设置好。	若为输出格式定义了已知设备则使用该设备，否则使用输出格式本身。
precommand	在实际转换命令之前执行的命令。	dvips \subjobname（DVI/PS），（PDF 下为空）
onfailure	设置转换失败时应触发的 ‘message’ 类型：error、warning、info 或 ignore。	warning

换设置来配置。Ghostscript 可执行文件在 Linux/Unix 下通常名为 'gs'，在 MS Windows 下名为 'gswin64c'（32 位版本为 'gswin32c'），并默认按此配置，但可以用 gsexe 设置更改。默认使用 64 位版本，可以用 ghostscript32 转换选项改为 32 位版本。

也支持转换程序 Image Magick,它可在 Unix/Linux、Mac 与 MS Windows 上免费获取。不过对 PDF 输入它内部仍会使用 *Ghostscript*，但增加了更多输出文件类型，如 GIF 格式。在 Ubuntu Linux 下可以用 shell 命令 'sudo apt-get install imagemagick' 安装。该转换可执行文件原先名为 'convert'，与 MS Windows 的 FAT 转 NTFS 转换器重名。较新的版本把可执行文件命名为 'magick'，此时转换命令必须作为参数给出：'magick convert'。如有需要，可以用 'convertexe' 转换选项手动更改可执行文件名，或在配置文件 'standalone.cfg' 中使用：

```
\standaloneconfig{convert={convertexe={...}}}
```

此外还支持矢量图形软件 Inkscape。它可以把 PDF 文件转换成 SVG（便于网页发布）和 EMF（便于把图形放进 MS Office）。它同样可在 MS Windows、Linux 与 macOS 上免费获取。

6.3 转换流程

转换流程目前的实现方式如下，目的是只做一次（手工的）编译运行就能完成正常编译和随后的转换。由于必须先完整编译文档才能转换，standalone 会把同一个 LaTeX 编译器（例如 pdflatex）作为子进程再执行一次，由它完整编译当前文档。这一步在载入 standalone 文档类时发生，以便主编译器实例仍停留在 \documentclass 处，尚未自己打开输出文件写入。文档经子进程编译完成后，就会执行外部转换工具。如有需要，诸如 dvips 的中间转换也会事先执行。最后，主编译运行在不产生任何输出的情况下结束，从而保持子进程生成的输出文件完好无损。这种实现的一个缺点是子进程创建的日志文件会被主进程覆盖，因而不含有效信息。可以用 subjobname 转换设置为子进程设置不同的 jobname 来弥补。

6.4 转换示例

生成 PNG（默认）图像，尺寸为 640px（适合上传到 StackExchange 站点而不被缩小）：

```
\documentclass[convert={size=640}]{standalone}
```

生成 EMF，这是一种矢量格式，适合插入 MS Word 或其他 MS Office 软件。文本内容会被渲染成曲线，无法再作为文本复制。此格式需要安装 *Inkscape*，因为其他转换软件不支持该输出格式。

```
\documentclass[emf]{standalone}
```

或

```
\documentclass[convert={emf}]{standalone}
```

生成 SVG，这是一种矢量格式，适合用于网站。文本内容会被渲染成曲线，无法再作为文本复制。此格式需要安装 *Inkscape*，因为其他转换软件不支持

该输出格式。

```
\documentclass[svg]{standalone}
```

或

```
\documentclass[convert={svg}]{standalone}
```

生成尺寸为 640px 的 GIF。这需要安装 *ImageMagick*，因为其他转换软件不支持该输出格式。

```
\documentclass[gif={size=640}]{standalone}
```

或

```
\documentclass[convert={gif,size=640}]{standalone}
```

把 PDF/PS 以 600x100dpi 光栅化，然后转换成 JPG（不推荐）：

```
\documentclass[convert={density=600x100,outext=.jpg}]{standalone}
```

生成 400x400px 的 BMP（若内容形状不是正方形，某一边可能会更小）：

```
\documentclass[convert={outext=.bmp,size=400}]{standalone}
```

用 Ghostscript 生成 TIFF G4 输出文件，密度为 72dpi：

```
\documentclass[convert={ghostscript,gsdevice=tiffg4,
                outext=.tiff,density=72}]{standalone}
```

7 standalone 宏包的用法

7.1 基本用法

只需在主文档中用 \usepackage 载入 standalone 宏包即可。它会重定义子文件里可能出现的 \documentclass 宏,使它忽略直到下一个 \begin{document} 的所有内容，然后把 document 环境当作一个简单的分组。主文件中真正的 document 环境不受影响。之后就可以在主体中用 \input{⟨ 文件名 ⟩} 读入子文件。

不能在文档类之前用 \RequirePackage 载入 standalone 宏包，因为这样做会导致问题。同样，也不能在导言区中 \input standalone 文件，例如作为 \savebox 赋值的一部分。

可以级联（cascade）standalone 文件，即在一个 standalone 文件内部 \input 另一个 standalone 文件。此时所有上级 standalone 文件都必须同时载入 standalone 文档类和 standalone 宏包。这些上级文件若遇到自己载入 standalone 宏包的其他 LaTeX 文档，仍然可以被使用。

关于可启用更多功能的宏包选项列表，见小节 7.2。

7.2 宏包选项

standalone 宏包支持下列选项。其中大部分是布尔选项，以 'true' 或 'false' 作为可选值。若这样的选项不带值使用，则采用 'true'。除非另有说明，所有选项初始都设为 'false'。选项可能会开启或关闭其他选项。选项的先后顺序会被遵守，后面的选项会压过前面的选项。请注意，standalone 宏包的某些旧版本只接受不带任何值的选项。

`subpreambles=true|false`

载入时，standalone 宏包默认会移除所有子文件导言区（"子导言区"）。不过，若带 subpreambles 选项载入该宏包，所有子导言区都会被存入一个名为 '⟨ 主 *TeX* 文件名 ⟩.sta' 的辅助文件（其中的 sta 表示 *sta*ndalone）。之后在下一轮 LaTeX 运行的开始处（即在导言区中载入 standalone 宏包的位置），该文件会被载入或处理。subpreambles 选项的工作方式可以由 sort、print 与 comments/nocomments 选项控制。请注意，sort 与 print 选项自然需要 subpreambles 选项，并在尚未启用时把它启用。

`sort=true|false`

若只设置了 subpreambles 选项，子导言区只是被简单地读入并原样执行。这就带来一种风险：同一个宏包在子导言区和/或主导言区中被带不同选项载入时会发生选项冲突。sort 选项可以避免这一点，它把所有子文件载入的所有宏包连同其选项累积起来。之后用 LaTeX 的 \PassOptionsToPackage 宏把这些选项标记为该宏包的选项。宏包在导言区末尾通过 \AtBeginDocument 钩子载入。这样用户就能在载入 standalone 宏包之后，在主文件中带自己的选项载入同样的宏包，而不会产生任何选项冲突。

`print=true|false`

虽然 sort 选项已经给出了不错的结果，但宏包顺序方面的问题仍可能出现。某些宏包会提供、重定义或修补相同的宏，因此必须按正确的顺序载入才能得到想要的结果。子导言区中可能还有额外代码，它们对某些子图是必需的，却可能与其他子图不兼容，这使情况更加复杂。若出现这类问题，很难用自动方式处理。此时必须把子导言区仔细地合并进主导言区。print 选项正是为了简化这项原本繁琐的工作而创建的。它把所有子导言区拼接到一个名为 '⟨ 主 *TeX* 文件名 ⟩.stp' 的单一文件中（其中的 st 表示 *st*andalone，p 表示 *p*rint）。每个导言区都会以其原始文件名作为注释。请注意，上面提到的 .sta 文件虽然相当类似，但含有额外的宏，可能不易于用户阅读或编辑。

文件生成之后，就可以用文本编辑器方便地把它粘贴进主文件的导言区。

启用 print 选项时，普通的 .sta 文件不会被生成或载入。由于这很可能导致一些与宏包未载入有关的错误，所有子文件主体都会被跳过。每个子文件都会打印一条警告以提醒用户这一事实。print 选项只在需要获取子导言区列表时才应当使用。把该列表包含进主文件之后，必须去掉该选项才能正常编译主文件。

`print,sort`

最后，若 print 与 sort 选项都启用，一份“排好序的”子导言区列表会被打印进 .stp 文件。在这种“排序打印”模式下，所有 \usepackage 宏以及其他类似的宏，如 \usepgflibrary 、\usetikzlibrary 以及 \usetikztiminglibrary（分别来自 pgf、tikz 与 tikz-timing 宏包），都会从子导言区的其余代码中移除。一份不含重复项的宏包（与库）列表会被打印在 .stp 文件的开头（当然是用 \usepackage ）。任何子文件为某个宏包提供的选项都会被加上，同样不重复。若某个子文件中要求了特定的宏包日期，该日期也会被加上。若同一个宏包被要求了多个日期，则采用最新（即“最高”的，而不是最后处理的）那个。在这些列表之后，子导言区的其余代码会在移除上述宏之后打印出来。这种模式免去了用户手动删除重复项、收集宏包选项的需要。

请注意，位于花括号 {} 之内的所有 \usepackage 及类似宏都不会被 standalone 的排序宏看到，因此不会被提取，也不会按上述任何特殊方式处理。可以利用这一点只在 standalone 模式下、而不在主文档中载入某些宏包。遗憾的是，位于 \ifstandalone…\fi 内部的宏会被看到并被提取，尽管在主文件中并不希望如此。正是由于这两个原因，才创建了宏 \onlyifstandalone{⟨ 代码 ⟩}（见下文）。它的参数花括号把内容对扫描器隐藏起来。因此它也会被完全从打印出的子导言区代码中移除。

`comments=true|false`
`nocomments`

comment 选项决定 .stp 文件是否也应包含子导言区的注释。为向后兼容而保留了 nocomments，它等同于 comments=false 。在非排序的打印模式（print 不带 sort 选项）下，注释默认会被包含，但它们可能在“排序”过程

中导致“错误”的结果，因此在该模式下默认被移除。原因可以这样解释。为了把子文件中的注释转移到.stp 文件，必须让 TEX 把它们当作普通输入处理而不丢弃。然而在这种情况下，移除\usepackage 及其同类的扫描算法就无法区分“活动的”宏与被注释掉的宏。注释中上述所有宏都会被当作“活动的”来处理。用户也许更看重注释所提供的信息，而不在乎这一小小的风险，于是用 comments 选项启用它们。

`group=true|false`

该选项默认设为 ‘true’，它控制是否在 standalone 文件的内容外加一个分组。通常（‘true’）子文件的 document 环境会变成一个除了加上通常的分组之外什么也不做的环境。若设为 ‘false’，该环境变得透明，因此不会加上分组。子文件文档主体内的任何定义在 \input 宏之后仍然可以访问。请注意，这不会影响 \includestandalone 宏，它总会加上一个分组。

`mode=⟨ 模式 ⟩`

设置 \includestandalone 的模式。有效值为 ‘tex’（使用源文件，默认）、‘image’（使用源文件生成的现有图像文件）、‘image|tex’（若有图像则用图像，否则用源文件）、‘build’（从源文件构建图像，然后使用它）、‘buildmissing’（仅在图像不存在时才构建图像）以及 ‘buildnew’（仅在源文件更新时才构建图像；在 XeLaTeX 下不起作用）。详见小节 7.3。另见小节 7.4 的进一步说明。

`obeyclassoptions=true|false`

若启用该选项，\includestandalone 在 ‘tex’ 模式下会尽量遵守 standalone 文件中所用的类选项。这仅在 standalone 文件使用 standalone 文档类、且只对某些选项时才有效。类配置文件也会被载入（对每个 standalone 文件都在一个局部作用域中），以便载入默认设置。

该功能的目的是确保把 standalone 文件作为源代码读入或作为图像读入时结果（几乎）相同，从而可以在两种模式之间轻松切换。具体来说，标准尺寸选项 10pt、11pt 与 12pt 会被应用于 standalone 文件（对标准文档类和 KOMA Script 文档类受支持），border 类选项也是如此。支持 multi’=’⟨ 环境 ⟩, … 选项，它会使 \includegraphics 的 page=⟨ 页码 ⟩ 选项对 \includestandalone 起作用。这意味着可以选定某一页，而跳过所有其他环境。默认取第一页（若使用了 multi）。特殊值-1 会读入源文件中的所有页面（但不含图像中的）。由于 multi 选项会假定 crop 或 preview 已启用，并且总像 ignoreresst=true 那样忽略其他内容。这三个类选项会被该宏包忽略，这可能导致 standalone 模式与主文档模式下的行为不同，但只在少数不常见的情形中出现：使用 multi 而未声明环境，且禁用了裁切（crop/preview）。为支持可能的 varwidth 选项，若 varwidth 可用则会被载入。

这是一个扩展功能，需要大量额外代码和一些高级技术来切换字号。它

未必在所有情况下都能正确工作。因此它默认被禁用。目前它不考虑类配置文件，也不适用于 beamer 的 standalone 文件。

`extension=`⟨. 扩展名⟩

用于 mode=image 的图像文件扩展名（含前导的点）可以用该选项选择。默认使用所用 LaTeX 编译器的目标输出扩展名，即 pdflatex、lualatex 与 xelatex 用 '.pdf'，latex 用 '.eps'（由 DVI 转换而来）。

`build={`⟨ 构建选项 ⟩`}`

该选项用于设置从 standalone 文件构建图像时所用的选项。详见小节 7.4，尤其是表 3。

7.3 宏

下列用户宏由 standalone 宏包提供。节 8 中列出了更多宏，它们由文档类与宏包共同定义，既可用于 standalone 文件，也可用于主文档。

`\standaloneconfig{`⟨ 选项 ⟩`}`

该配置宏接受小节 7.2 中描述的部分宏包选项。这些选项是 group、mode、extension 与 build，它们可以针对不同的被读入 standalone 文件分别更改。

若同时使用了 standalone 文档类与宏包，该宏也可以用来设置小节 5.3 中描述的类选项。

`\includestandalone[`⟨ 选项 ⟩`]{`⟨ 文件 ⟩`}`

这个功能强大的宏可以代替 \input 来读入 standalone 文件。它的行为由 mode 宏包选项控制。该宏既可以像 \input 那样读入源代码（mode=tex），也可以用 \includegraphics 读入输出文件（PDF、EPS）（mode=image），或先试输出文件、若有源文件则用源文件（mode=image|tex），或从源文件构建输出文件：总是构建（mode=build）、仅在图像文件不存在时构建（mode=buildmissing）或仅在源文件更新时构建（mode=buildnew，它在 XeLaTeX 下不起作用，因为这一功能需要 pdfTeX 的一些宏）。更多细节另见小节 7.4。

⟨ 文件 ⟩ 参数必须是 standalone 源文件的文件名，且不含扩展名。该宏接受与 \includegraphics 相同的 ⟨ 选项 ⟩，以及任何适用于 \standaloneconfig 的选项。这意味着源文件在 'tex' 模式下也可以像图像一样被缩放和旋转。待办：在此模式下，该宏包还会尝试提取 standalone 文件中的类选项，并把它们应用于读入的源代码。遗憾的是，无法完全保证 standalone 内容在源代码模式与图像模式下显示得完全一致。某些设置可能不会以同样的方式生效，还可能产生舍入差异。

7.4 从 standalone 文件构建图像

使用 `\includestandalone` 宏，standalone 文件既可以作为源代码直接读入，也可以作为由这些源文件构建出的矢量图形图像读入。`standalone` 宏包提供了从给定的 standalone 源文件自动构建图像文件的功能。这由 mode 选项控制。前面已在小节 7.2 与 7.3 中描述过。

这使用户可以轻松地在读入源代码与读入图像之间切换，既可全局切换，也可只针对选定的 standalone 文件切换。使用图像的好处是，被读入的材料（通常是比较复杂的图片）不必每次都随主文档重新编译。这会带来显著的速度提升。缺点是文件体积会略微增大，因为该材料会有自己的文件头。另外，主文档中任何会影响源代码的设置都不会对图像起作用。这究竟是好是坏视情况而定。

一个扩展功能是从 standalone 文件自动构建图像，可以总是构建，也可以仅在源文件比现有图像文件更新时才构建。在这些情况下，`\includestandlone` 宏会对相应的 standalone 文件调用 LaTeX 编译器生成图像，然后用 `\includegraphics` 读入它们。这需要设置 '`-shell-escape`' 编译器选项，否则出于安全原因会禁止执行 shell 命令。

图像文件通常会在主文档的当前目录中生成，而该目录未必与源文件所在的目录相同。取决于所用的编译器设置，当前目录中的文件会先于其他目录被找到。因此，使用 `mode=buildnew` 时，放在当前目录中新构建的图像文件会先于可能位于 standalone 文件目录中的旧图像文件被采用。由于在 LaTeX 文档内部无法获知源文件的确切目录，因此不可能总把图像文件生成在与源文件相同的目录中。诸如 '`-output-directory`' 的编译器选项可用于影响构建图像的输出目录。不过这些选项必须用于内部的编译运行，即通过适当地设置 `build={latexoptions=⟨...⟩}`，而不是（仅仅）用于主 LaTeX 编译运行。

若构建过程失败，会发出警告，并改为读入源代码。应当指出，失败检测并不完美，可能出现误报或漏报。

8 共用的宏

下列条件宏由 `standalone` 文档类与宏包共同定义，但代码单独编译或作为主文档一部分编译时行为不同。

`\ifstandalone`

文档类与宏包都提供 if 开关 `\ifstandalone`，可用于只在文件单独编译时才包含的代码。该开关被文档类设为 `\iftrue`，被宏包设为 `\iffalse`。

附加文件 `standalone.tex` 也会通过检查 `\documentclass` 是否已被使用来定义该开关。它可以用`\input{standalone}` 读入，专供不使用 `standalone` 文档类的特殊文件使用。

表 3: 构建设置

构建设置	说明	默认值
`latex`	要使用的 LaTeX 编译器	与主编译器相同
`latexoptions`	编译器的命令行选项	`-interaction=batchmode -shell-escape -jobname \quote \buildjobname \quote \file`
`jobname`	构建编译运行使用的 jobname	`\file`
`quote`	用于给文件名加引号的字符	Linux & Mac OS X 用 `'`，Windows 用 `"`
`command`	完整的构建 shell 命令	`\latex \space \latexoptions \space \file`
`postcommand`	在主命令之后执行、用于生成最终输出文件的命令	`dvips -o '\file.eps' '\file.dvi'`（仅 DVI 模式）

注意：这些设置（'command' 与 'postcommand' 除外）也可以在其他设置中作为宏使用。给定的文件名（不含扩展名）可以通过 `\file` 取得。

`\IfStandalone{`⟨*standalone* 模式下的代码⟩`}{`⟨ 主文档中的代码 ⟩`}`

这是 `\ifstandalone` 开关的宏版本。它只在 `standalone` 模式下（即文件单独编译时）执行第一个参数。当被读入主文档时，改为执行第二个参数。如小节 7.2 所述，它也可以用来对 `sort` 选项的提取扫描器隐藏`\usepackage` 及类似宏。该宏及其参数不会被打印进`.stp` 文件。

`\onlyifstandalone{`⟨ 代码 ⟩`}`

该宏与 `\IfStandalone` 类似，但只接受一个参数，该参数只在 standalone 模式下执行，作为主文档一部分编译时则被忽略。如小节 7.2 所述，它也可以用来对 `sort` 选项的提取扫描器隐藏`\usepackage` 及类似宏。该宏及其参数不会被打印进`.stp` 文件。

9 用法示例

示例 1: *standalone* 宏包的用法。

```
% 主文件
% 真正的文档类:
\documentclass{article}

% 使用 'standalone' 宏包:
\usepackage{standalone}

% 载入所有子文件需要的全部宏包:
\usepackage{tikz}

% 在真正的 'document' 环境内部
% 用 '\input' 读入子文件
\begin{document}
% ...
\begin{figure}
  \input{subfile}
  \caption{一个子文件}
\end{figure}
% ...
\end{document}
```

示例 2: *standalone* 文档类的用法。

```
% 一个使用 'standalone' 文档类的子文件（例如一张图片）:
% 用 'standalone' 作为文档类:
\documentclass{standalone}

% 载入本 TeX 文件需要的宏包:
\usepackage{tikz}

% 像平常一样把 TeX 代码置于 'document' 环境之中:
\begin{document}
% 在此添加你的 TeX 代码，例如一张图片:
\begin{tikzpicture}
  \draw (0,0) rectangle (2,1) node [midway] {示例};
\end{tikzpicture}
\end{document}
```

示例 3: 单独编译时实际生效的代码。

```
\documentclass{article}

\newenvironment{standalone}{\begin{preview}}{\end{preview}}
\input{standalone.cfg}
% 它默认会载入:
% \PassOptionsToPackage{active,tightpage}{preview}
\usepackage{preview}

\usepackage{tikz}

\begin{document}
\begin{standalone}
\begin{tikzpicture}
  \draw (0,0) rectangle (2,1) node [midway] {示例};
\end{tikzpicture}
\end{standalone}
\end{document}
```

示例 4: 被包含进主文档时实际生效的代码。

```
\begingroup
\begin{tikzpicture}
  \draw (0,0) rectangle (2,1) node [midway] {示例};
\end{tikzpicture}
\endgroup
\endinput
```